第三章 SQL 注入漏洞与防护——实验分析报告

任务 1：数据库与查询条件基础说明

1）页面输入为什么会和数据库查询有关？

DVWA 的这个页面，核心功能是 “根据用户输入的 ID，查询并展示对应的用户信息（姓名、ID 等）”。这些用户信息（比如 ID、姓名）都存储在后端的 MySQL 数据库里，网页本身不保存这些数据，只能通过向数据库发送 SQL 查询指令，获取到数据后再渲染到页面上。所以，用户在页面输入框里填的内容，会直接影响数据库要执行什么查询、返回什么数据。

2） 最小业务链路是什么？

以 DVWA 的用户查询为例，用户输入 ID（比如 1） → 网页拼接 SQL 语句（比如 SELECT * FROM users WHERE id='1'） → 数据库执行查询 → 把查到的用户信息返回给网页 → 网页渲染结果显示给用户

3）用户输入是如何进入查询条件的？

网页后端会把用户的输入直接拼接到 SQL 语句里。用户输入的内容，会被当成 SQL 语句的一部分，和固定的 SQL 代码拼在一起，最终被数据库执行。

4）为什么这为 SQL 注入创造了可能性？

因为后端没有把 “用户输入的数据” 和 “SQL 指令本身” 分开处理，而是把用户输入直接当成 SQL 语句的一部分来执行。正常情况下，用户输入的只是 “数据”（比如 1），但如果用户输入的内容里包含了 SQL 语法（比如 1’ OR 1=1#），后端会把它当成 SQL 语句的一部分拼接进去。

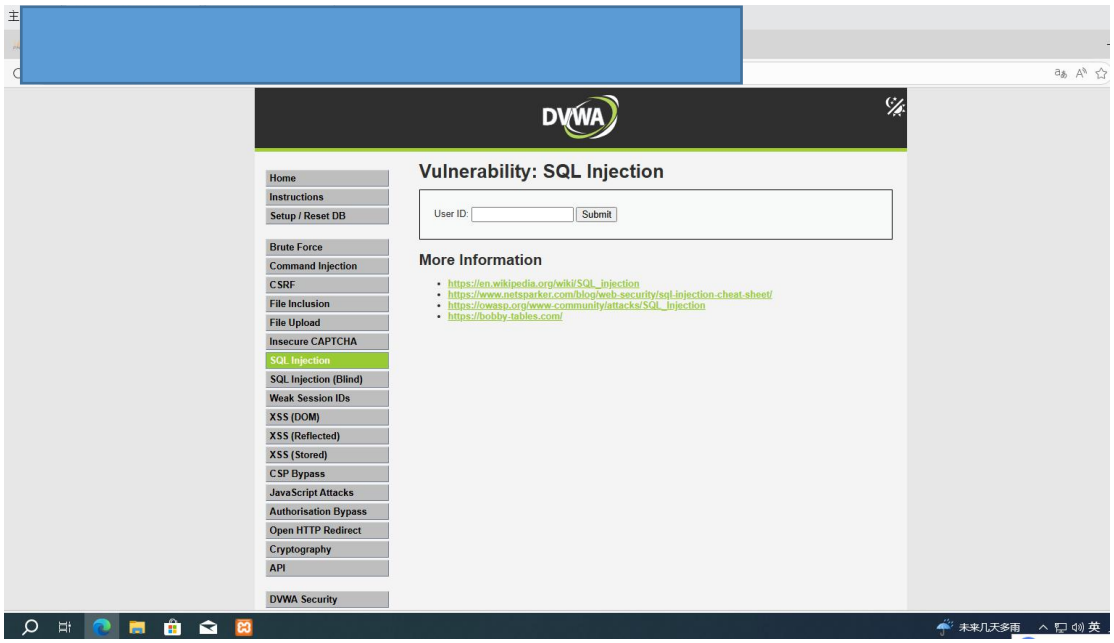
业务场景举例：生成的 SQL 会变成：SELECT first_name, last_name FROM users WHERE id = '1' or 1=1# '，改变了原本的查询逻辑， ' 闭合了原有的单引号，让输入脱离了 “字符串数据” 的范畴；or 1=1 让查询条件永远为真；# 注释掉了后面的所有语句，避免语法错误。

最终数据库会返回 users 表里的所有用户数据，这就是 SQL 注入的核心，用户输入污染了 SQL 语句的结构，改变了原本的查询逻辑。

任务 2：DVWA SQL Injection Low 现象记录

1）实验页面 / 实验目标

实验环境为本地 DVWA 平台（地址：127.0.0.1:8081/dvwa/），实验目标是「SQL Injection」模块，安全级别设置为 Low。实验目标是观察用户输入对数据库查询结果的影响，验证 SQL 注入漏洞的存在，并分析现象差异。



2）普通输入与异常输入的对比

普通输入：输入 1

输入内容：1

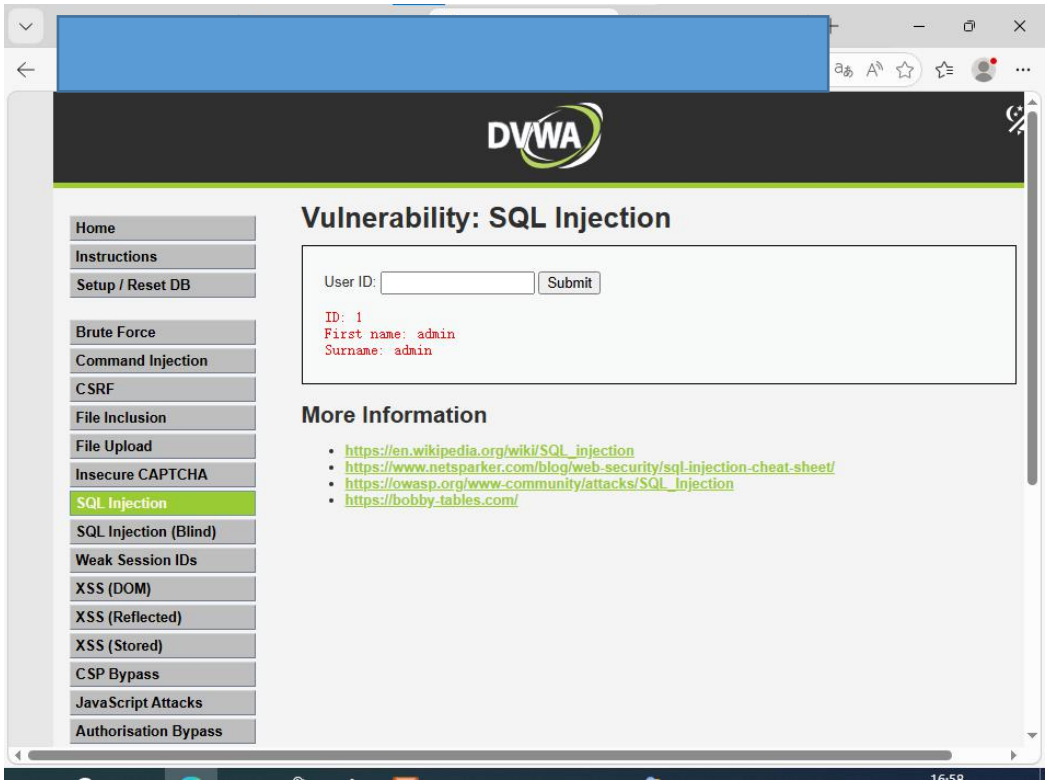
现象：页面仅返回 ID 为 1 的用户信息：

ID: 1

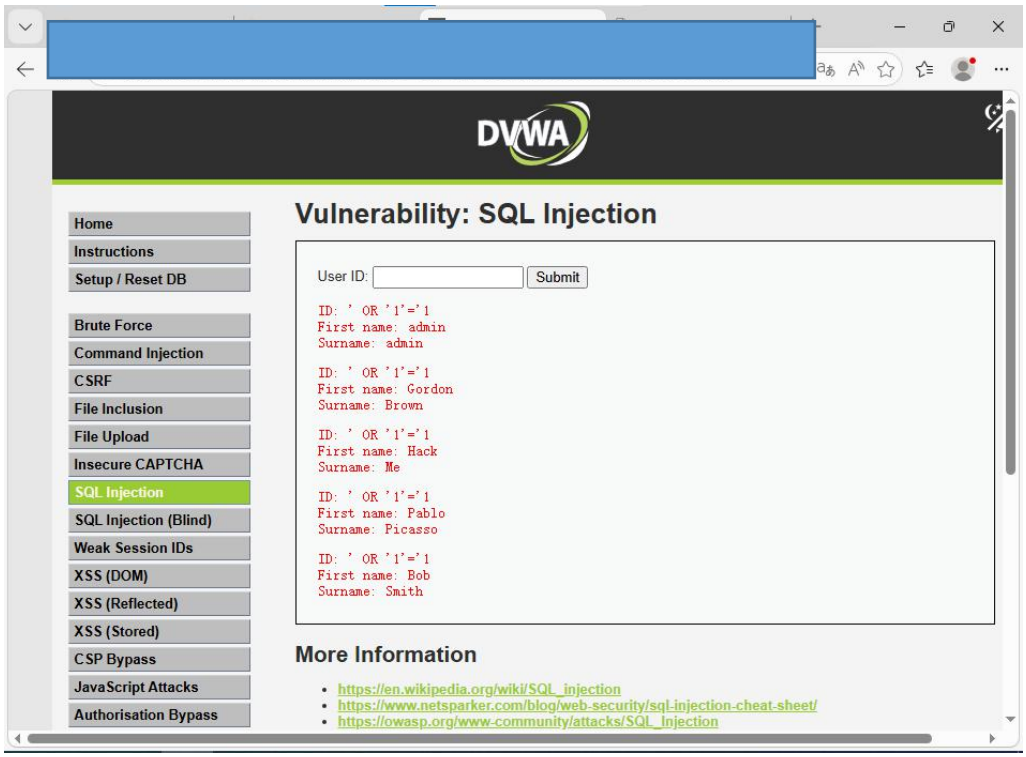
First name: admin

Surname: admin

结论：正常输入下，查询逻辑是「按 ID 精准匹配，返回单个用户信息」。



异常输入输入 ' OR '1'='1（或你用的 1' OR 1=1#）
输入内容：' OR '1'='1
现象：页面返回了数据库中所有用户的信息
结论：异常输入下，查询逻辑被篡改，返回了全表数据。



3）为什么这种差异不像 “正常查询变化”？

正常的查询结果变化，只会在输入不同的合法 ID 时，返回对应的单条用户数据，结果和输入的 ID 是一一对应的。而这次异常输入的结果，存在两个完全不符合正常业务逻辑的地方：输入的内容本身不是合法的 ID（包含单引号、OR 等非数字字符），但数据库没有报错，反而执行了查询；输入一个值，却返回了数据库里的所有用户数据，这是业务逻辑不可能出现的情况，说明查询条件被用户输入篡改了，不是正常的查询变化。

任务 3：机制分析与差异比较

1）完整的执行链条

以 Low 级别的注入为例：用户输入（比如' OR 1=1 #）→ 后端拼接（把输入拼进 SQL 语句，生成 SELECT * FROM users WHERE id='' OR 1=1 #'）→ 数据库解析（MySQL 把拼接后的语句当成合法的 SQL 指令解析，识别出 OR 1=1 的条件）→ 数据库执行（执行查询，返回所有满足条件的记录）→ 行为差异（页面返回所有用户信息，和正常查询结果完全不同）

2） 为什么根因是 “结构被污染”，而不是 “字符本身危险”？

很多人会觉得'、OR、--、#这些字符是危险的，但其实这些字符本身没有问题：比如你在数据库里写 SELECT * FROM users WHERE id='1' OR 1=1，这是一个完全合法的 SQL 语句

真正的问题是：用户输入的内容，被当成了 SQL 语句的 “结构部分”，而不是 “数据部分”后端没有把用户输入当成 “单纯的字符串数据”，而是直接拼进了 SQL 语句，让用户可以修改查询的逻辑结构，这才是 SQL 注入的根因。就算没有'，如果后端用的是数字型 SQL 语句，输入 1 OR 1=1 也能实现注入，所以 “字符本身” 不是问题，“结构被污染” 才是。

3） Low 与 Medium 级别的差异

对比项	Low	Medium
后端处理	直接把用户输入拼进 SQL 语句，无任何过滤	对用户输入做了 mysql_real_escape_string() 转义，单引号被转义为\'
注入难度	非常容易，直接拼接即可注入	单引号被转义，无法直接用字符串型注入，需要用数字型注入
注入方式	字符串型注入（需要用'闭合单引号）	数字型注入（输入 ID 不带引号，比如 1 OR 1=1）
核心差异	没有任何输入处理	对字符串做了转义，但如果后端用的是数字型查询，转义就失效了

4）普通 SQL 注入与盲注的差异

对比项	普通 SQL 注入（有回显）	盲注
信息获取方式	直接通过页面回显获取数据（比如 UNION 查询直接返回数据库版本）	没有直接回显，只能通过页面的 “正常 / 异常” 状态、响应时间来判断数据
结果证据形式	页面直接返回查询结果（比如数据库版本、用户名）	页面状态差异（比如输入正确的条件时页面正常，错误时页面报错 / 空白；或基于时间延迟的差异）
注入难度	低，能直接看到结果	高，需要逐位猜解数据
根因	输入污染了 SQL 结构	输入污染了 SQL 结构（和普通注入的根因完全一样）

5）为什么盲注和普通注入根因相同，但证据形式不同？

两者的根因都是用户输入污染了 SQL 语句的结构，改变了原本的查询逻辑，区别只在于 “结果是否直接回显”：

普通注入：数据库执行结果直接返回给页面，用户能直接看到数据，所以证据是 “页面回显的差异”

盲注：数据库执行结果不会直接返回，只能通过页面的间接状态（比如是否报错、是否延迟）判断查询结果，所以证据是 “页面行为的差异”，但背后的 SQL 结构被污染的逻辑是完全一样的。

6）强制判断题

SQL 注入的根因更接近

选 B：输入进入了不该进入的查询结构

理由：根因是用户输入被当成了 SQL 语句的结构部分，而不是数据部分，污染了原本的查询逻辑；危险字符只是实现注入的手段，不是根因。

Blind SQL Injection 和普通 SQL 注入的最大差异更接近

选 B：证据形式不同

理由：两者的根因都是输入污染了 SQL 结构，差异只在于结果的表现形式（一个直接回显，一个通过页面状态间接判断），根因是完全相同的。

任务 4：SQL 注入防护清单与解释

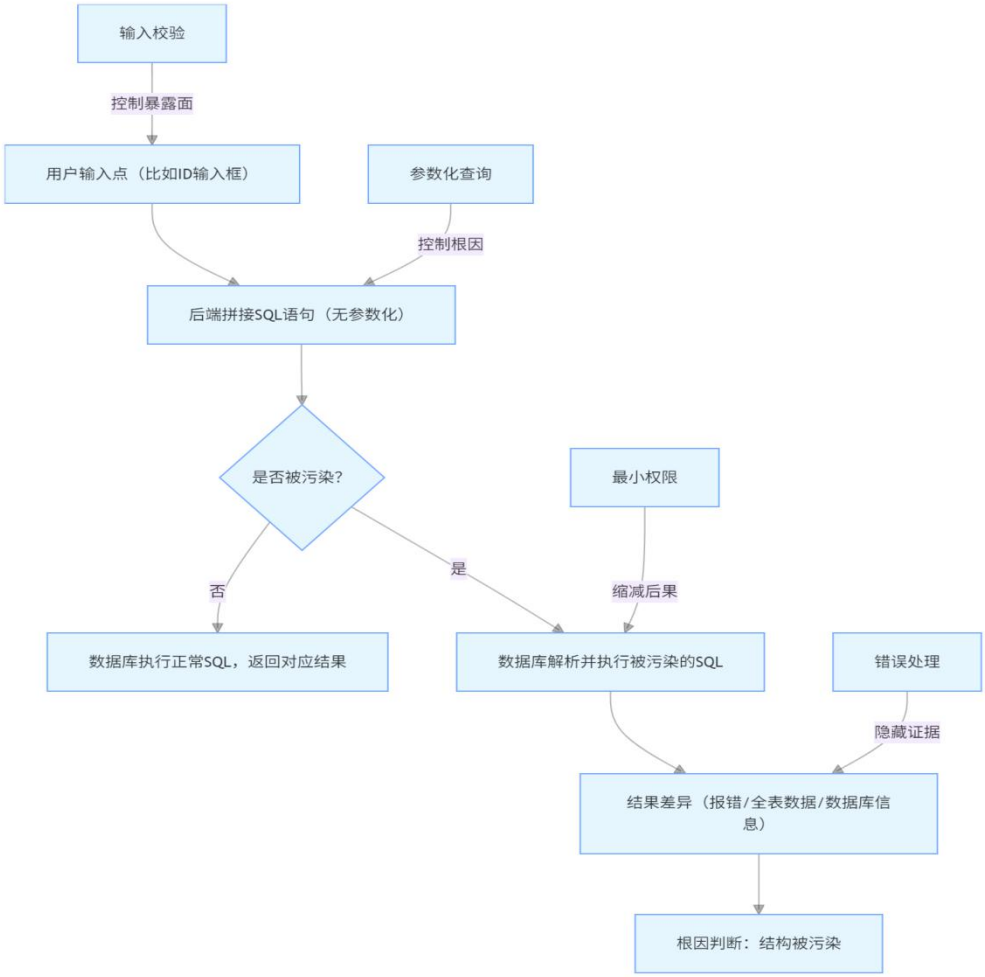
防护清单

防护措施	**解决的问题**	**控制层级**	**不能替代的措施**
1. 参数化查询	从根上区分 “SQL 指令结构” 和 “用户输入数据”，用户输入永远只会被当成数据处理，不会	根因控制（直接解决注入的根本	不能替代输入校验（参数化也无法过滤恶意的业务输入，

防护措施	解决的问题	控制层级	不能替代的措施
	被解析成 SQL 结构，彻底避免结构污染	问题）	比如超出范围的 ID）
2. 输入校验	限制用户输入的格式和范围，比如 ID 只能是数字、长度固定，过滤掉不符合格式的输入，减少注入的可能性	暴露面控制（减少可能被利用的输入点）	不能替代参数化查询（就算校验了，也可能被绕过，无法从根上解决结构污染）
3. 错误处理	关闭页面的详细数据库错误回显，避免攻击者通过报错信息判断数据库结构、表名、字段名，减少攻击的线索	暴露面控制（隐藏敏感信息，降低攻击成功率）	不能替代参数化查询（只是隐藏了线索，无法阻止注入本身）
4. 最小权限	数据库连接账号只分配业务必需的权限，比如只给查询用户表的权限，不分配增删改、查询系统表的权限，就算被注入，也无法获取额外数据或破坏数据库	后果缩减（限制注入造成的影响范围）	不能替代根因控制（如果注入成功，还是能获取该权限下的所有数据，只是减少了损失）
5. 补充：（Web 应用防火墙）	识别并拦截常见的注入攻击流量（比如包含’OR 1=1 的请求），作为额外的防护层	暴露面控制（在应用层外增加一道拦截）	不能替代参数化查询（WAF 规则可能被绕过，无法从根上解决问题）

任务 5：SQL 注入因果链/防护关系图

（参考 ai)



任务 6：自评

我本章最清楚的一点是什么？

我最清楚的是 SQL 注入的根因 —— 用户输入污染了 SQL 语句的结构，而不是危险字符本身，所有防护措施的核心目标都是避免结构被污染。

我最容易混淆的一点是什么？

我最容易混淆的是普通注入和盲注的差异，一开始以为它们的根因不同，后来才明白两者的根因都是结构污染，只是证据形式不同。

我本次作业中哪一段最需要教师复核？

任务 3 里关于 Low 和 Medium 级别的差异分析，以及普通注入和盲注的差异对比，我不确定自己对“根因”的理解是否准确。

如果重做一次，我会优先改哪一部分？

我会优先完善任务 2 和 5 补充更多的注入测试截图，并在每张截图后更详细地解释现象和结论的对应关系，以及自己画出流程图。

任务 7：AI 输出审核与人工复核记录

1.本作业哪些内容使用过 AI 辅助？

本次作业中，基础概念解释、因果链梳理、防护清单，我使用了 AI 辅助理解；流程图的结构参考了 AI 的示例；部分文字的表述优化使用了 AI 辅助。

2.哪些判断是你自己复核后保留的？

SQL 注入的根因是“结构被污染”而非“危险字符”，普通注入和盲注的根因相同，差异在证据形式防护措施分为根因控制、暴露面控制、后果缩减三类这些判断我都结合 DVWA 的实验现象进行了验证，确认和实际结果一致后保留。

3.哪些内容你发现 AI 输出不准确并进行了修正？

AI 一开始把盲注的根因写成了“数据库没有回显”，我结合实验现象修正为“根因和普通注入一样，都是结构被污染，只是结果不直接回显”

4. 你最终如何确认结论和证据是一一对应的？

我把每一个结论都和自己在 DVWA 里的实验截图对应起来：比如“输入' OR 1=1 #返回所有用户”的结论，对应了我自己测试的截图；其次文字描述一字一句修改且改成自己能理解的意思。最后整体格式进行梳理美化。

.